# Java Programming 3

## Week 10: Spring Data JpaRepository

KdG

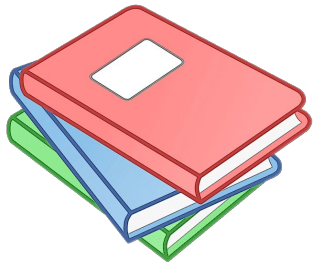
# Agenda this week

| |
|---|
| Spring data: JpaRepository |
| Query methods - Pagination - @Query |
| Inheritance and JPA |
| Service Layer Transactions |
| Project |

KdG

# Tutorials

- Spring Data Quick start: https://spring.io/guides/gs/accessing-data-jpa/
- Baeldung
  - Spring Data JPA: https://www.baeldung.com/the-persistence-layer-with-spring-data-jpa
  - Query Methods: https://www.baeldung.com/spring-data-derived-queries
  - @Query annotation: https://www.baeldung.com/spring-data-jpa-query



KdG

# Agenda this week

| **Spring data: JpaRepository** |
| --- |
| Query methods - Pagination - @Query |
| Inheritance and JPA |
| Service Layer Transactions |
| Project |

KdG

# Spring Data: automatic repository implementation!

```
public interface BookRepository extends JpaRepository<Book, Integer> {

}
```

We create a Repository interface that extends the JpaRepository<T,K> interface
T: the entity type
K: the type of the id

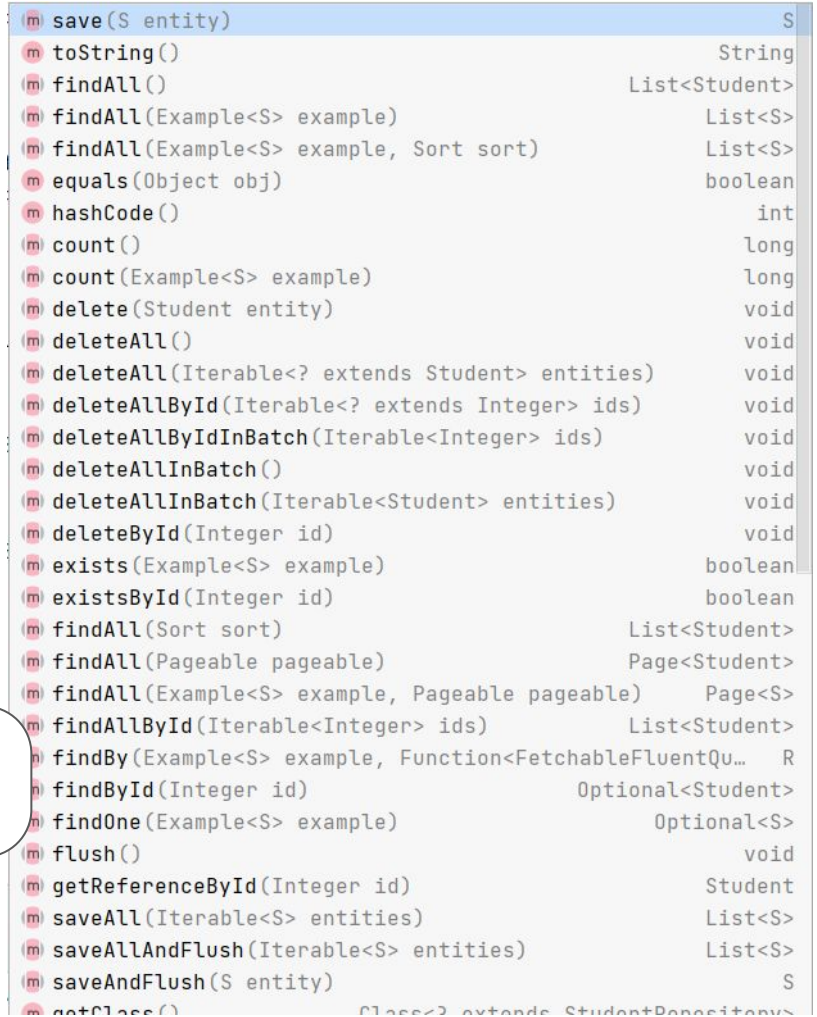
Spring automatically generates the implementation of the JpaRepository methods!

KdG

# JpaRepository methods

- save → save or update
- findAll → returns List
- deleteById
- exists
- findById
- …

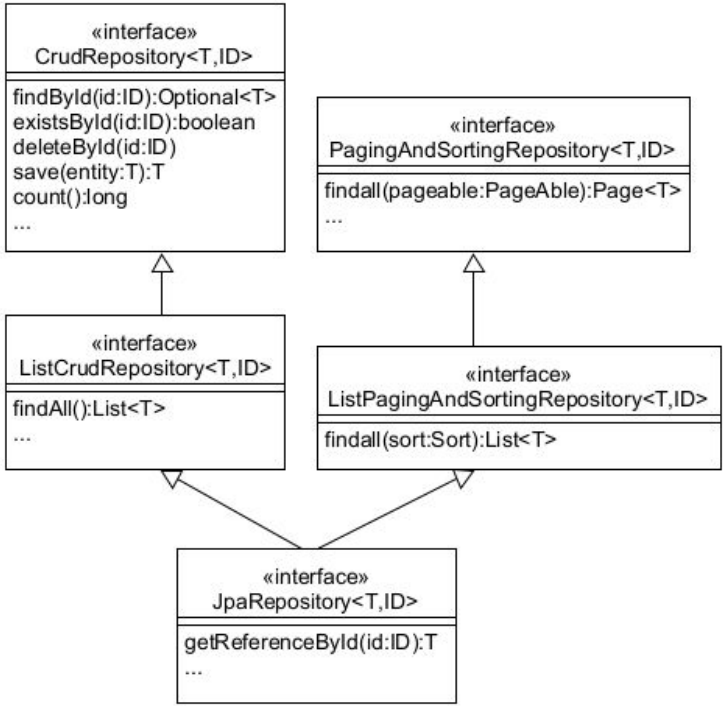
You get all these methods for free!




KdG

# JpaRepository inherits from other interfaces

- JpaRepository API Doc
- Inherits from
  - ListCrudRepository,CrudRepository:
    - Interface for generic CRUD operations
  - ListPagingAndSortingRepository:
    - methods to retrieve entities using the pagination and sorting abstraction
  - QueryByExampleExecutor
    - Interface to allow execution of Query by Example




KdG

# JpaRepository uses generics to define the Entity and the ID type

```
@Repository
public interface StudentRepository extends JpaRepository<Student, Integer> {
}
```

- Spring will generate the implementation of all those methods based on the generic types of the entity and the id

KdG

# Exercise: create StudentRepository

- Start from the startproject on gitlab: https://gitlab.com/kdg-ti/programming-3/exercises/studentjparepository
- Add a StudentRepository using JpaRepository
- Implement the StudentService method *addStudent*
- In the Demo class:
  - Add 100 students to the database (using the factory method in Student)
    - Check in h2-console!

KdG

# Exercise: elaborate the StudentService class

- Implement the other StudentService methods
  - findStudents
  - findStudent
  - changeStudent
  - deleteStudent
- In the Demo class:
  - Test the different methods!

KdG

# Agenda this week

| |
|---|
| Spring data: JpaRepository |
| **Query methods - Pagination - @Query** |
| Inheritance and JPA |
| Service Layer Transactions |
| Project |

KdG

# Query Methods

- If you need extra methods, you can add them to the interface. Spring Data JpaRepository will try to derive the implementation…
- For example:

If you follow the conventions, Spring is able to generate the code for these methods!

```
public interface StudentRepository extends JpaRepository<Student, Integer>
{
   List<Student> findByBirthDayAfter(LocalDate localDate);
   List<Student> findByAddressCity(String city);
}
```

Joins on Address

For more examples of the Query Methods syntax

- https://www.baeldung.com/spring-data-derived-queries
- Spring Query Methods User Guide

KdG

# Query Methods

```
public interface StudentRepository extends JpaRepository<Student, Integer> {
   List<Student> findByBirthDayAfter(LocalDate localDate);
   List<Student> findByFirstName(String firstName);
}
```

- Derived method names have two main parts separated by the first By keyword
  - The first part — such as find — is the **introducer**, and the rest — such as ByName — is the **criterium**.
- Introducers:
  - find, read, query, count, get
  - You can add Distinct, First or Top to limit the result (eg. findTop3ByAge)
- Criteria:
  - Entity-specific condition expressions:
    - keywords along with entity's property names
    - Concatenate with And, Or

KdG

# Query Methods

- Equality Condition:
  - findByName(String name) and equivalents
    - findByNameIs(String name)
    - findByNameEquals(String name)
  - findByNameIsNot(String name)
  - Nulls:
    - findByNameIsNull()
    - findByNameIsNotNull()
  - Booleans:
    - findByActiveTrue()
    - findByActiveFalse()

KdG

# Query Methods

- Similarity Condition:
  - findByNameStartingWith(String prefix)
  - findByNameEndingWith(String suffix)
  - findByNameContaining(String infix)
  - Patterns:
    - findByNameLike(String likePattern)
      - Example pattern: “a%b%c” (starts with ‘a’, contains ‘b’ and ends with ‘c’

KdG

# Query Methods

- Comparison Condition:
  - findByAgeLessThan(Integer age)
  - findByAgeLessThanEqual(Integer age)
  - findByAgeGreaterThanEqual(Integer age)
  - findByAgeGreaterThanEqual(Integer age)
  - findByAgeBetween(Integer start, Integer end)
  - findByAgeIn(Collection<Integer> ages)
  - findByBirthDateAfter(LocalDate date)
  - findByBirthDateBefore(LocalDate date)

KdG

# Query Methods

- Multiple Condition: combine with And and Or
  - findByNameOrAge(String name, int age)
  - findByNameOrBirthDateAndActive(...)
- Sorting the results:
  - findByNameOrderByName(String name)
  - findByNameOrderByNameAsc(String name)

KdG

# Exercise: workout the StudentService class

- Add and implement the following StudentService methods (and add the Query Methods to the StudentRepository…)
  - findBigStudents() //length>200
  - findSmallStudents() //length<140
  - findYoungStudents() //younger than 10 years
  - findStudentsFromPothoekStraat() //address starts with PotHoekStraat
  - findTenFolds() //name ends with 0
- In the Demo class:
  - Test the different methods!

KdG

# @Query annotation: define the query yourself…

- Default: uses JPQL (Java Persistance Query Language, defined in JPA standard)
  - You can also use SQL: use the nativeQuery = true attribute

```
@Query(value = "SELECT * FROM USERS u WHERE u.status = 1", nativeQuery = true)
Collection<User> findAllActiveUsersNative();
```

- Sorting: we can add a Sort parameter to the method (only for JPQL):

```
@Query(value = "SELECT u FROM User u")
List<User> findAllUsers(Sort sort);
```

  - Usage:

```
userRepository.findAllUsers(Sort.by("name"));
```

KdG

# @Query annotation

- Pagination
  - Return a subset in a Page → useful if there are many results
    - use to page in results in your web application!
- Example (more examples):

```
@Query("SELECT s FROM Student s ORDER BY s.id")
Page<Student> findAllStudentsWithPagination(Pageable pageable);
```

- Usage:

```
studentRepository.findAllStudentsWithPagination(PageRequest.of(0, 20));
```

You can also add Sort info to a PageRequest

KdG

# @Query annotation

```
@Repository
public interface StudentRepository extends JpaRepository<Student, Integer> {
 List<Student> findByFirstName(String firstName);

 List<Student> findByBirthDayBefore(LocalDate date);

 @Query("""
   SELECT s
   FROM Student s
   JOIN FETCH s.courses
   WHERE s.id = :id
  """)
  Optional<Student> findByIdWithCourses(int id);

 @Query("SELECT s FROM Student s WHERE s.address.city = :city")
 List<Student> findByCity(String city);

}
```

JPQL JOIN FETCH will eagerly load a relation

On OneToOne relations you can use attribute chaining instead of JOIN

KdG

# Exercise: elaborate the StudentService class

- Add and implement the following StudentService methods: use a custom query on the repository
  - Add a method that finds a student by id and then show the student and his courses
  - findAdults();//age >= 18
  - findNamesOfStudentsThatAreNotUnique()
  - find Students that are in the Programming Course
- Query for all students but return pages of size 20, use pagination…
- In the Demo class:
  - Test the different methods!

KdG

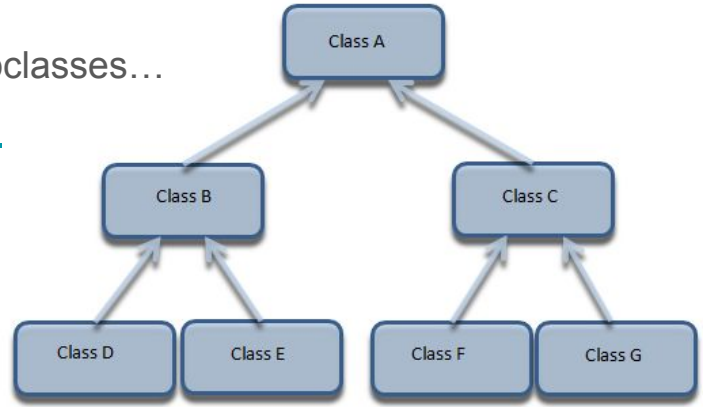
# Agenda this week

| |
|---|
| Spring data: JpaRepository |
| Query methods - Pagination - @Query |
| **Inheritance and JPA** |
| Service Layer Transactions |
| Project |

KdG

# How to map Inheritance to the database?

- Different strategies
  - MappedSuperclass – the parent classes are no entitites
  - Single Table – The entities from different classes with a common ancestor are placed in a single table.
  - Joined Table – Each class has its table, and querying a subclass entity requires joining the tables.
  - Table per Class – All the properties of a class are in its table, so no join is required.
- Polymorphic behaviour:
  - You can query for the superclass and will retrieve all subclasses…
- https://www.baeldung.com/hibernate-inheritance




KdG

# MappedSuperclass

- You inherit from a super class which is not an entity
  - its fields are not  saved to the database
  - annotations on the super class have no effect
- You need to annotate it with @MappedSuperclass to save fields and inherit annotations (e.g. @id)

```
@MappedSuperclass
public class Person {

    @Id
    private long personId;
    private String name;

    // constructor, getters, setters
}
```

```
@Entity
public class Employee extends Person {
    private String company;
    // constructor, getters, setters
}
```

- Results in database table for Employee with 3 columns (personId (PK), name, company)

KdG

# Single Table

- Inheritance tree is mapped to one table
  - This is the default strategy, you can omit @Inheritance here

```
@Entity
@Inheritance(strategy =
InheritanceType.SINGLE_TABLE)
public class Product {
    @Id
    private long id;
    private String name;

    // constructor, getters, setters
}
```

```
@Entity
public class Book extends Product {
    private String author;
}
```

```
@Entity
public class Pen extends Product {
    private String color;
}
```

- Results in database table PRODUCT (id, name, author, color, dtype)
- DTYPE contains discriminator value = name of entity (Book, Pen)
  - You can choose different discriminator columns and values with annotations…

KdG

# Joined Table

- Each class is mapped to a different table.
- Each table has the **same** id column, and for child classes it is PK **and** FK to parent class.

```
@Entity
@Inheritance(strategy =
InheritanceType.JOINED)
public class Animal {
    @Id
    private long animalId;
    private String species;

    // constructor, getters, setters
}
```

```
@Entity
public class Pet extends Animal {
    private String name;

    // constructor, getters, setters
}
```

- Results in 2 tables: ANIMAL(animalid, species) and PET(animalid, name)
  - animalid in PET is PK and FK to ANIMAL
- You have to perform JOINS to collect data of PET…

# Table per Class

- Each entity is mapped to a different table. Each table has all the properties of the entity, including the inherited ones.

```
@Entity
@Inheritance(strategy =
InheritanceType.TABLE_PER_CLASS)
public class Vehicle {
    @Id
    private long vehicleId;
    private String manufacturer;

    // standard constructor, getters, setters
}
```

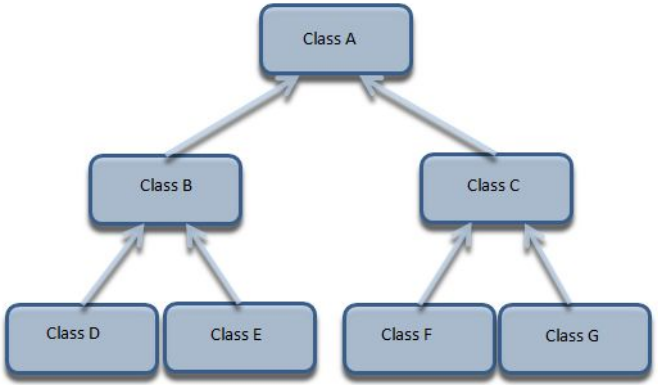
```
@Entity
public class Bike extends Vehicle {
    private String type;

    // constructor, getters, setters
}
```

- Results in 2 tables: VEHICLE(vehicleid, manufacturer) and BIKE(vehicleid, manufacturer, type)
    - If you query the superclass, JPA will perform a UNION in the background to load all subclasses

KdG

# Exercise: add inheritance




- Add 2 subclasses for Student:
  - ACSStudent(country) and FlexStudent(occupation)
- Use the default strategy: Single Table
- Startup the application and inspect the database
- Add some ACSStudents and FlexStudents and experiment!

KdG

# Agenda this week

| |
|---|
| Spring data: JpaRepository |
| Query methods - Pagination - @Query |
| Inheritance and JPA |
| **Service Layer Transactions** |
| Project |

KdG

# @Transaction in Service Layer

- Often the service layer methods will contain business logic
- This can involve taking a few queries to one or more repositories that should be managed by a transaction
- You can use the @Transactional annotation on top of these Service layer methods
  - Spring will ensure that these steps are done in one transaction.
  - All entities created and loaded are part of the same persistence context

KdG

# @Transaction in Service Layer

- An example: Student and Book
  - Student has a List of Books (1-to-many relation)
  - The Books are lazy fetched (configured in the Student entity)
  - Presentation layer needs the books of a certain student

In presentation layer:

```
student = studentService.findStudent(1).get();
System.out.println(student);
System.out.println(student.getBooks());
```

This will not work, because the books are lazy fetched!

KdG

# Exercise: convert JDBC repo to Spring Data Repository

- Start from a JDBC Template implementation of the relationsdemo: https://gitlab.com/kdg-ti/programming-3/exercises/relationsdemo_with_jpa
- Now try to convert this demo to a Spring Data JPA implementation using JpaRepositories:
  - Introduce a Service layer, for @Transactional methods
  - Think about your Cascades!
  - Think about your Fetch strategies!

KdG

# Agenda this week

| |
|---|
| Spring data: JpaRepository |
| Query methods - Pagination - @Query |
| Inheritance and JPA |
| Service Layer Transactions |
| **Project** |

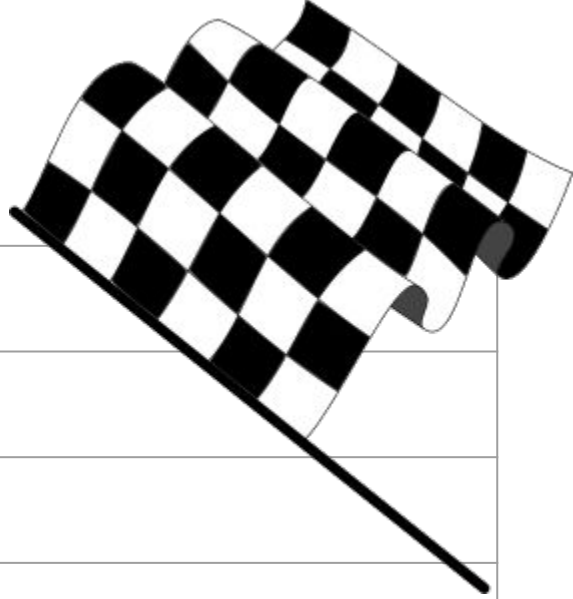
KdG

# Project

- In application.properties disable open session in view
  `spring.jpa.open-in-view=false`
- Add a subclass to one of your entities. Integrate it in the screens of one of your entities and in your DB code.
- Create a fourth implementation of your repository: use Spring Data JpaRepository
  - Because JpaRepository is another interface next to your existing repository interface, you have 2 options (you can choose):
    - Create an implementation of your interface that delegates the calls to the JpaRepository methods
    - Or create another Service layer implementation that calls the JpaRepository method
- Add *at least two method queries* to your main entity's repository and use them in your application (you can create extra pages for them)
- Add *at least one custom query method* (using @Query annotation) and use it in your application

KdG

# Agenda this week



| |
|---|
| Spring data: JpaRepository |
| Query methods - Pagination - @Query |
| Inheritance and JPA |
| Service Layer Transactions |
| Project |

KdG